Uber Dash - API de Estadísticas de Viajes

Moises Agudelo Campo

Samuel Muñoz Quiroga

Plataformas Computacionales

Carlo Tello

Pontificia Universidad Javeriana Cali

Marzo 24 2026

2. INTRODUCCIÓN

Uber Dash es una API REST desarrollada con NestJS y TypeScript que permite gestionar usuarios, vehículos y viajes, con el objetivo de generar estadísticas y mapas de calor de recorridos.

Esta API está diseñada para funcionar como un dashboard de análisis de datos de movilidad, similar a Uber pero sin la funcionalidad de contacto entre conductores y pasajeros. Su enfoque principal es el almacenamiento y consulta de información geoespacial para la visualización de rutas y zonas de mayor actividad.

La aplicación resuelve la necesidad de tener una plataforma centralizada donde se puedan registrar viajes, asociarlos a usuarios y vehículos, y posteriormente consultar métricas como viajes por período, zonas más concurridas y comportamientos de movilidad.

3. OBJETIVOS

Objetivo General

Desarrollar una API REST funcional que permita la gestión de usuarios, vehículos y viajes para el análisis de datos de movilidad.

Objetivos Específicos

- Implementar operaciones CRUD para usuarios, vehículos y viajes.
- Establecer relaciones entre entidades (usuario → vehículos → viajes).
- Implementar validaciones de datos de entrada como capa de seguridad.
- Utilizar SQLite como base de datos embebida para facilitar la instalación.
- Exponer endpoints claros y documentados para consumo del frontend.

4. DESCRIPCIÓN DE LA API

Característica	Descripción
Tipo de API	REST
Arquitectura	MVC (Model-View-Controller) con módulos
Lenguaje	TypeScript
Framework	NestJS v11
Base de Datos	SQLite (archivo .aqlite)
ORM	TypeORM
Validación	class-validator + ValidationPipe

Estructura General del Proyecto

```
src/
|----main.ts              # Punto de entrada
|---- app.module.ts          # Módulo principal
|----config/              # Configuración (variables de entorno)
|---- user/               # Módulo de usuarios
|---- car/                # Módulo de vehículos
|---- trip/               # Módulo de viajes
|___test/                 # Pruebas end-to-end
```

5. ENDPOINTS

5.1 Usuarios (/users)

Método	Ruta	Descripción
POST	/users	Crear un nuevo usuario

| GET | /users | Listar todos los usuarios |

| GET | /users/:id | Obtener un usuario por ID |

| PATCH | /users/:id | Actualizar un usuario |

| DELETE | /users/:id | Eliminar un usuario |

Ejemplo de Request (POST /users):

```json
{
  "name": "Juan Pérez",
  "email": "juan@example.com",
  "password": "123456"
}
```

Ejemplo de Response:

```json
{
  "id": 1,
  "name": "Juan Pérez",
  "email": "juan@example.com",
  "createdAt": "2026-03-21T10:00:00.000Z"
}
```

5.2 Vehículos (/cars)

Método	Ruta	Descripción
POST	/cars	Registrar un vehículo
GET	/cars	Listar todos los vehículos
GET	/cars/:id	Obtener un vehículo por ID
PATCH	/cars/:id	Actualizar un vehículo
DELETE	/cars/:id	Eliminar un vehículo

Ejemplo de Request (POST /cars):

```json
{
  "brand": "Toyota",
  "model": "Corolla",
  "plate": "ABC123",
  "userId": 1
}
```

Ejemplo de Response:

```json
{
  "id": 1,
  "brand": "Toyota",
  "model": "Corolla",
  "plate": "ABC123",
  "user": {
```

```
    "id": 1,
    "name": "Juan Pérez"
  }
}
```

5.3 Viajes (/trips)

Método	Ruta	Descripción
POST	/trips	Registrar un viaje
GET	/trips	Listar todos los viajes
GET	/trips/:id	Obtener un viaje por ID
GET	/trips/user/:userId	Obtener viajes de un usuario
PATCH	/trips/:id	Actualizar un viaje
DELETE	/trips/:id	Eliminar un viaje

Ejemplo de Request (POST /trips):

```json
{
  "startLat": 4.6097,
  "startLng": -74.0817,
  "endLat": 4.6523,
  "endLng": -74.0702,
  "distance": 5.2,
  "duration": 15,
  "userId": 1,
  "carId": 1
```

```
}
```

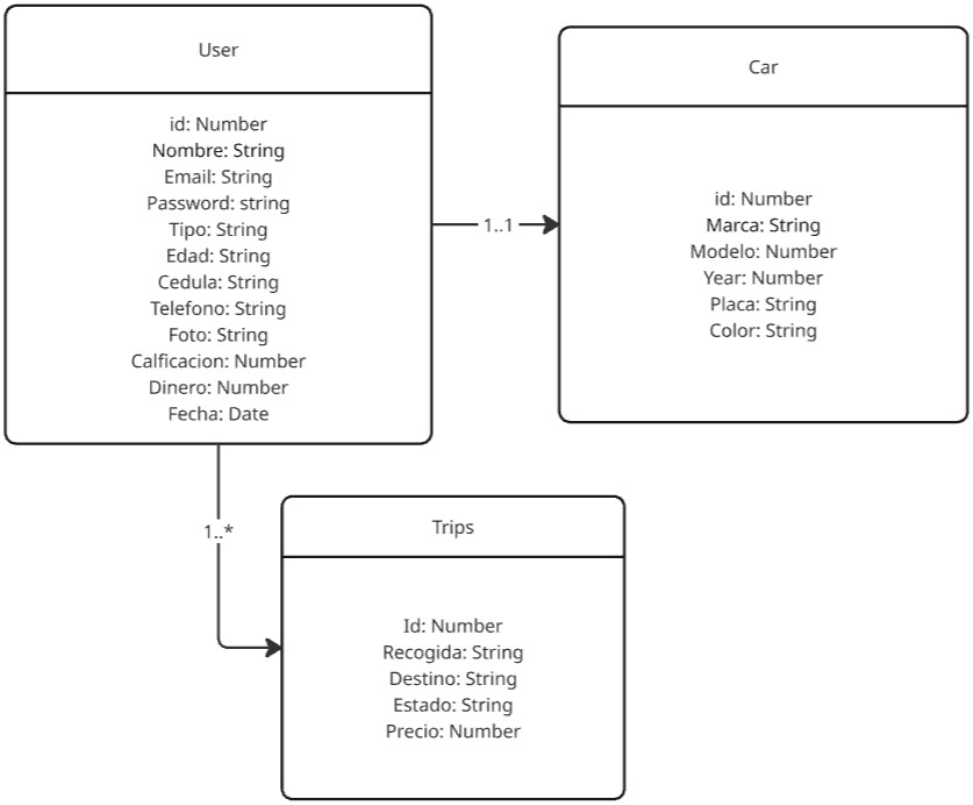
Ejemplo de Response:

```json
{
  "id": 1,
  "startLat": 4.6097,
  "startLng": -74.0817,
  "endLat": 4.6523,
  "endLng": -74.0702,
  "distance": 5.2,
  "duration": 15,
  "user": {
    "id": 1,
    "name": "Juan Pérez"
  },
  "car": {
    "id": 1,
    "brand": "Toyota",
    "plate": "ABC123"
  }
}
```

6. BASE DE DATOS

Tipo de Base de Datos: SQLite (archivo .sqlite local)

Diagrama de Relaciones



Configuración

La base de datos se configura automáticamente mediante TypeOrmModule en app.module.ts con las siguientes opciones:

- synchronize: true (crea tablas automáticamente)

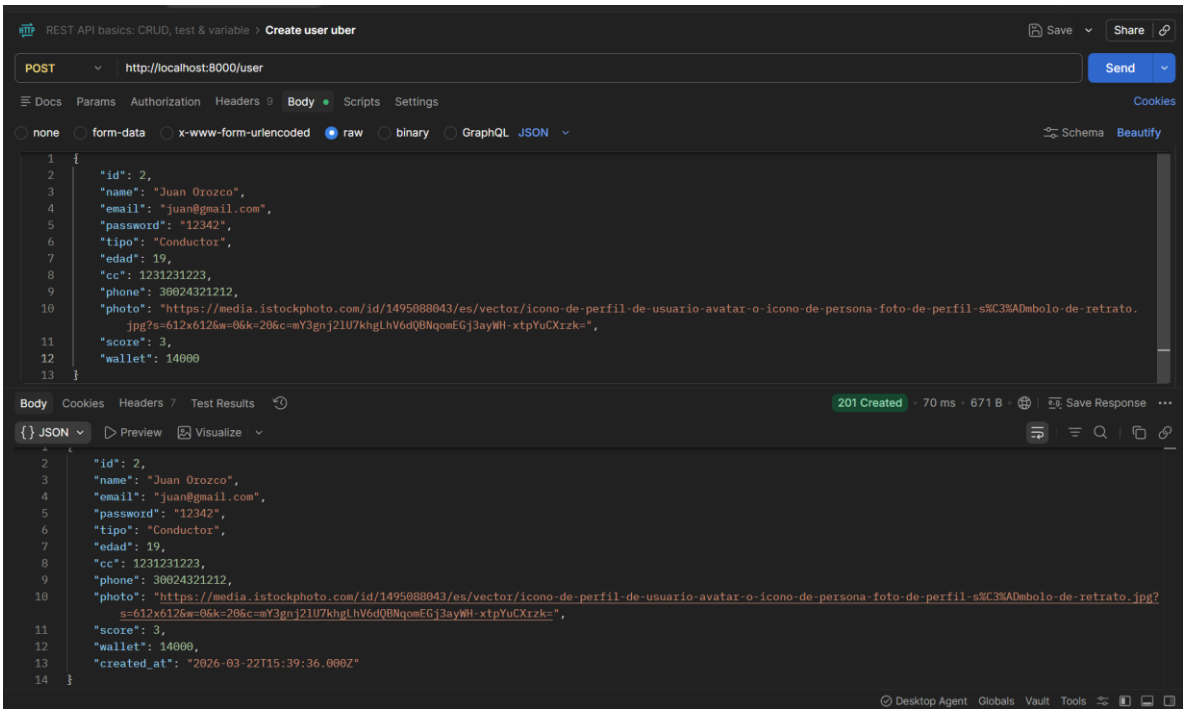
- database: 'database.sqlite' (archivo en la raíz)

7. EVIDENCIAS (CAPTURAS)

7.1 API funcionando en navegador

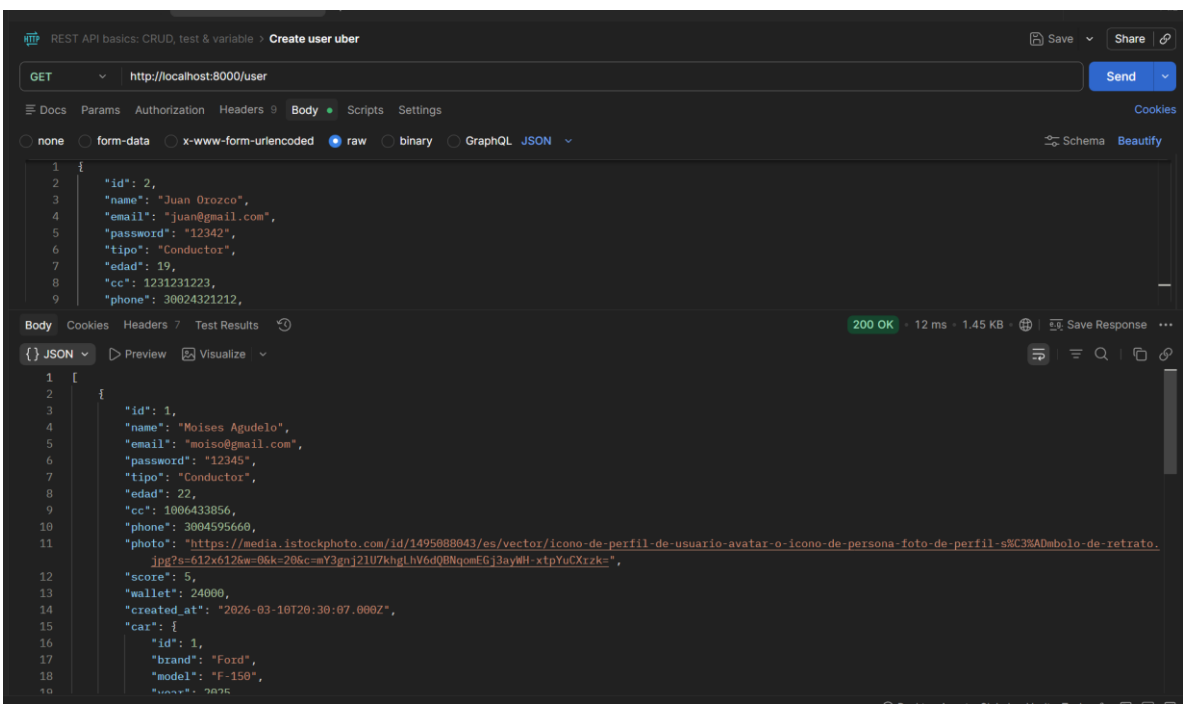
[captura de pantalla de http://localhost:8000 mostrando "Hello World!"]

7.2 Prueba de creación de usuario en Postman



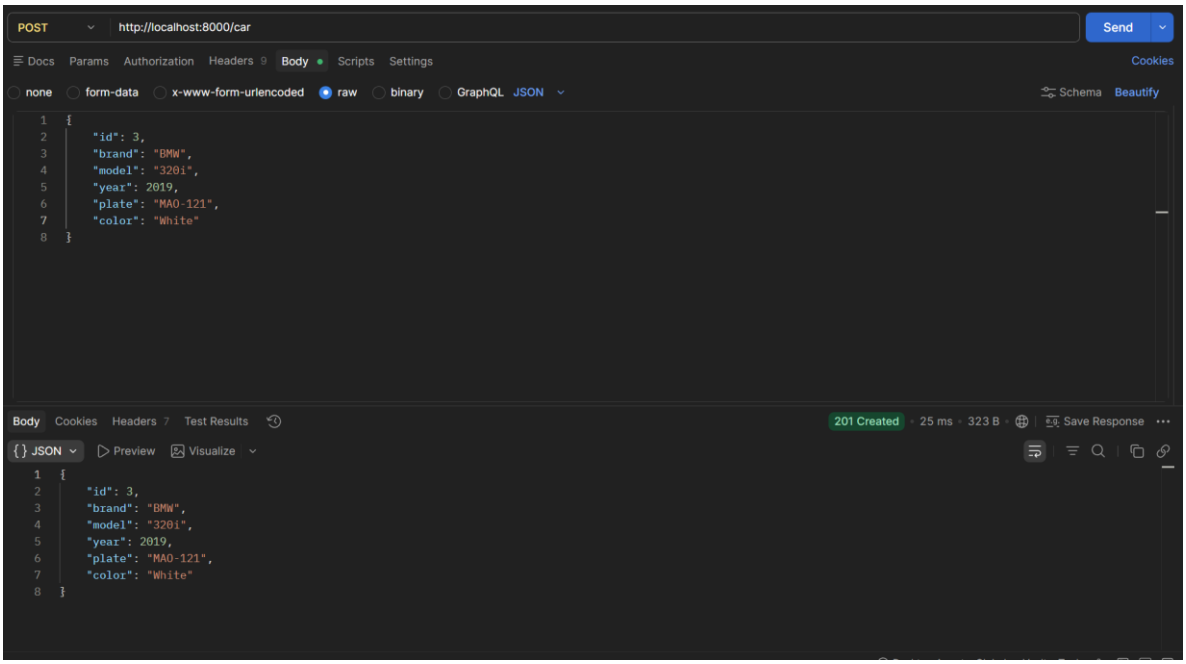
[captura de POST /users con response 201 Created]

7.3 Prueba de listado de usuarios



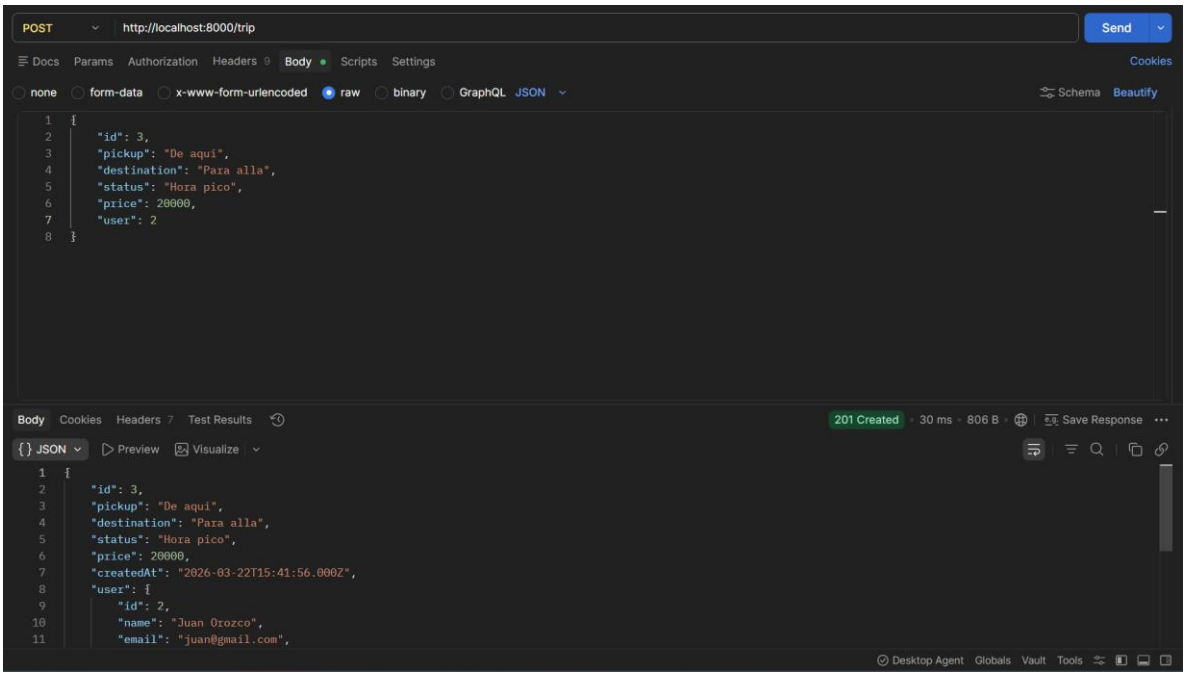
[captura de GET /users con lista de usuarios]

7.4 Prueba de creación de vehículo



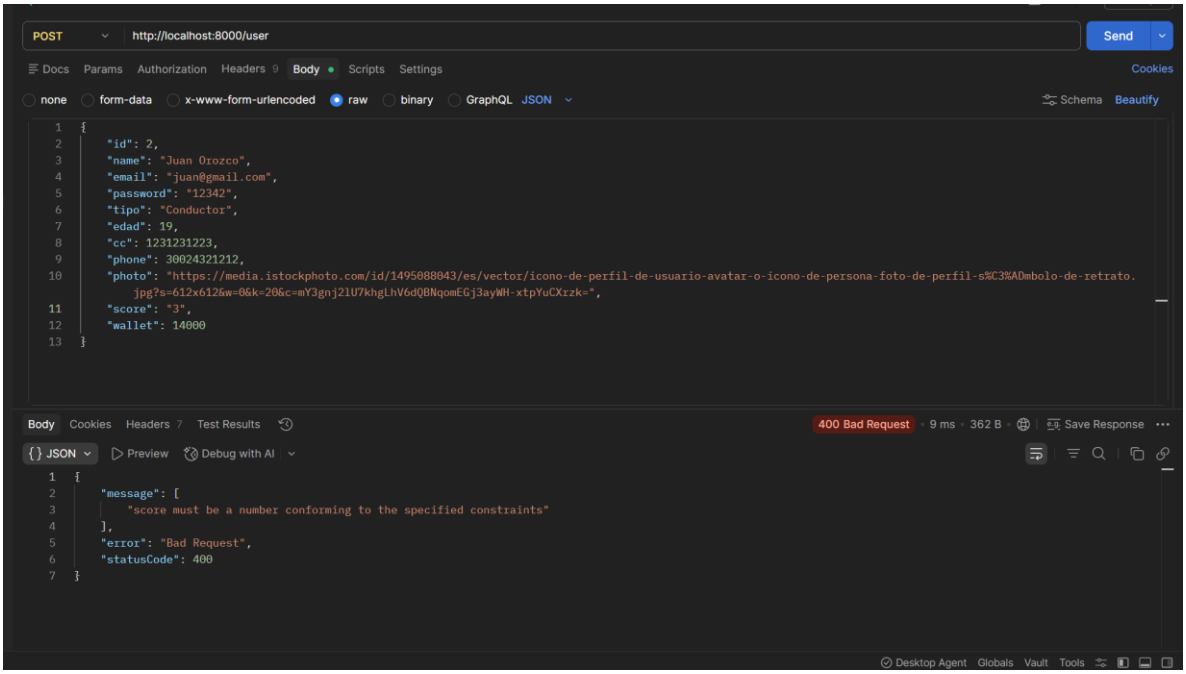
[captura de POST /cars con response]

7.5 Prueba de creación de viaje



[captura de POST /trips con response]

7.6 Validación de errores



[captura de envío de datos inválidos con response 400 Bad Request]

7.7 Código destacado (ValidationPipe en main.ts)

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { ValidationPipe } from '@nestjs/common';

import { ConfigService } from '@nestjs/config';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    }),
  );

  const config = app.get(ConfigService);
  const port = config.get<number>('port') || 8000;

  await app.listen(port);
  console.log('App corriendo en el puerto ' + port);
}
bootstrap();
```

[Insertar captura del archivo main.ts mostrando la configuración del ValidationPipe]

8. LINK DEL GITHUB

El código fuente completo del proyecto se encuentra en el siguiente repositorio:

https://github.com/Elcale1703/uber_app.git

Contenido del repositorio:

- Código fuente completo en TypeScript
- Archivos de configuración (nest-cli.json, tsconfig.json)
- Dependencias definidas en package.json

- Pruebas unitarias y e2e
- Este mismo reporte en formato PDF
- Instrucciones de instalación en README.md

9. INSTRUCCIONES DE USO

Requisitos previos

- Node.js (v18 o superior)
- npm (v9 o superior)

Pasos para ejecutar la API

1. Clonar el repositorio

```bash
git clone https://github.com/tuusuario/uber_app.git
cd uber_app
```

2. Instalar dependencias

```bash
npm install
```

3. Configurar variables de entorno

Crear archivo .env en la raíz:

```env
# Aplicación
APP_PORT=8000
APP_ENV=development
```

```
# Base de datos
DB_TYPE=sqlite
DB_DATABASE=database.sqlite
# Seguridad
JWT_SECRET=produccion_secret_uber_app_nestjs
JWT_EXPIRES=3600s
```

4. Iniciar la API

```bash
npm run start:dev
npm run start:prod
```

5. Probar la API

- Abrir navegador en http://localhost:8000
- Usar Postman o cURL para probar los endpoints

Ejemplo de prueba rápida con cURL

Crear usuario

```
curl -X POST http://localhost:8000/users \
  -H "Content-Type: application/json" \
  -d '{"name":"Juan","email":"juan@test.com","password":"123456"}'
```

Ver usuarios

```
curl http://localhost:8000/users
```

10. CONCLUSIONES

Aprendizajes

- Se comprendió el funcionamiento de NestJS y su arquitectura basada en módulos.
- Se implementaron correctamente las validaciones de entrada con class-validator como capa de seguridad.
- Se establecieron relaciones entre entidades usando TypeORM.
- Se utilizó SQLite como base de datos embebida, facilitando la portabilidad del proyecto.

Dificultades encontradas

- Configuración inicial de TypeORM con SQLite requirió ajustes en los decoradores de entidades.
- Manejo de relaciones anidadas en las respuestas de los endpoints.
- Configuración del ValidationPipe global para que funcione con todos los DTOs.

Mejoras futuras

- Implementar autenticación con JWT para proteger los endpoints.
- Agregar endpoints específicos para estadísticas (viajes por período, zonas más concurridas).
- Integrar con servicios de mapas (Google Maps, Mapbox) para visualización de rutas.
- Dockerizar la aplicación para facilitar el despliegue.
- Implementar paginación en los endpoints de listado.

Documento elaborado por: Samuel Muñoz Quiroga / Moises Agudelo Campo

Fecha de entrega: Marzo 24 2026